

MONGODB

INTRODUCTION

A short, classroom-friendly introduction from Database → Atlas → Mongoose.

The complete picture

Client → Node.js/Express → Mongoose → MongoDB Atlas → MongoDB

Topics

01. Database & MongoDB
02. Atlas, AWS & Cluster
03. Compass
04. Mongoose & Connection
05. Complete Flow
06. Create Atlas Cluster
07. Connect Node.js
08. Quick Revision

01 — Database & MongoDB

Database: A system used to persist, organize, retrieve and update application data.

Why do we need a database?

Data stored only in a Node.js variable disappears when the server restarts. A database keeps data persistent.

MongoDB is a document-oriented NoSQL database.

Traditional	MongoDB
Table	Collection
Row	Document
Column	Field

```
{
  name: "Aarav",
  age: 20,
  city: "Bhopal"
}
```

02 — Atlas, AWS & Cluster

MongoDB Atlas is MongoDB's managed cloud service. It hosts MongoDB deployments so you don't have to install and manage MongoDB on your laptop/server.

Where does AWS come in?

AWS is a cloud provider. Atlas can run MongoDB on AWS infrastructure. **You normally do not create an EC2 machine just to use Atlas.**

Cluster / Deployment

The MongoDB environment your application connects to.

```

Your Node.js App
  |
  | connection string
  v
MongoDB Atlas
  |
  v
MongoDB Cluster
  |
  +-- Database
      +-- Collection
          +-- Documents
```

03 — MongoDB Compass

Compass is a desktop GUI for MongoDB. It helps you visually inspect databases, collections and documents.

Atlas	Compass
Cloud service	Desktop GUI
Creates/manages deployment	Explores data
Provides connection details	Connects using URI

Remember

Compass is **not the database**. It is a visual client used to connect to and explore MongoDB.

Typical connection:

```
mongodb+srv://username:password@cluster.mongodb.net/myDB
```

04 — Mongoose & Connection

Mongoose is an ODM for MongoDB and Node.js. It provides schemas, models, validation and convenient database APIs.

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema({
  name: String,
  age: Number
});

const User = mongoose.model("User", userSchema);
```

Connect to Atlas:

```
require("dotenv").config();

mongoose.connect(process.env.MONGODB_URI);
```

Important

Keep the MongoDB URI in **.env**. Never commit database credentials to GitHub.

05 — Proper Application Flow

```
graph TD
    User --> React[React / Frontend]
    React -- "HTTP request" --> Express[Express / Node.js]
    Express --> Controller[Controller / Service]
    Controller --> Mongoose[Mongoose]
    Mongoose --> Driver[MongoDB Driver]
    Driver --> Atlas[MongoDB Atlas]
    Atlas --> MongoDB[MongoDB]
    MongoDB --> Response[Response]
    Response --> Frontend[Frontend]
```

Golden rule

Frontend does NOT directly connect to MongoDB. The backend controls database access, authentication, validation and business logic.

Basic CRUD with Mongoose:

Create	User.create(data)
Read	User.find()
Update	User.findByIdAndUpdate()
Delete	User.findByIdAndDelete()

06 — Create a MongoDB Atlas Cluster

The exact Atlas UI may change, but the setup flow is essentially:

1. Create / sign in to MongoDB Atlas
2. Create a Project
3. Create a deployment / cluster
4. Select a cloud provider such as AWS
5. Select a nearby region
6. Choose the appropriate free/Flex/dedicated option
7. Create the deployment

Then configure security:

8. Create a Database User
9. Add your IP address to Network Access / IP Access List
10. Click **Connect** and choose your connection method

AWS ≠ Atlas

AWS provides cloud infrastructure. **Atlas** manages MongoDB on that infrastructure.

After the cluster is ready, you can connect it to Compass or your Node.js application.

07 — Connect Node.js + 08 — Revision

Install:

```
npm install express mongoose dotenv
```

.env:

```
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/myDB
```

Connection:

```
const mongoose = require("mongoose");

mongoose.connect(process.env.MONGODB_URI)
  .then(() => console.log("MongoDB Connected"))
  .catch(err => console.error(err));
```

Quick Revision

Database → stores persistent data

MongoDB → document database

Atlas → managed cloud MongoDB service

AWS → cloud provider

Compass → GUI client

Mongoose → Node.js ODM

URI → tells the application how to connect

Final mental model:

React → **Express** → **Mongoose** → **MongoDB Atlas** → **MongoDB**